TREES

Dr. Priyanka N

Assistant Professor Senior Grade 1

School of Computer Science & Engineering

VIT, Vellore.

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	**Linear Data Structures**	**7 hours**
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	**Searching and Sorting**	**7 hours**
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	**Trees**	**6 hours**
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		
Module:5	**Graphs**	**6 hours**
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	**Hashing**	**4 hours**
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	**Heaps and AVL Trees**	**5 hours**
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		

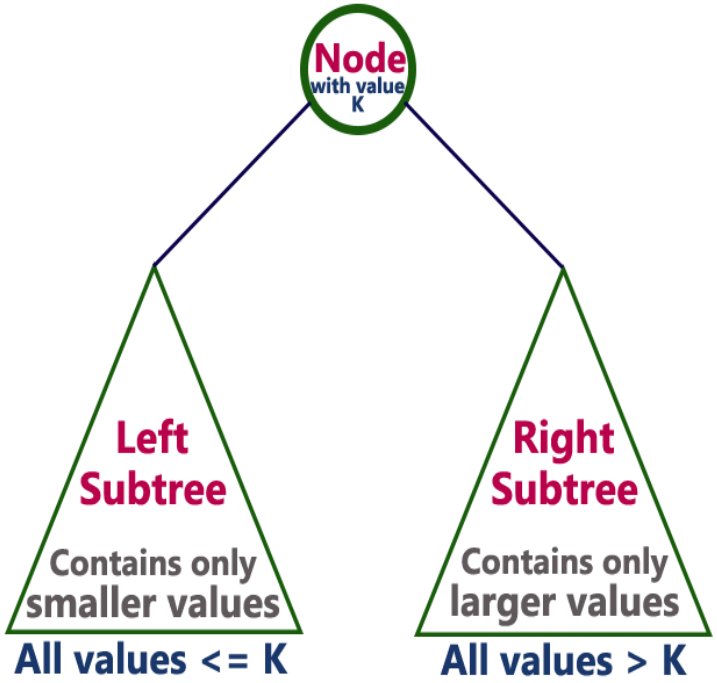
TREES

- Introduction to Trees & Terminologies
- Binary Tree – Definition and Properties
- Binary Tree Representation
- Tree Traversals
- Expression Trees
- **Binary Search Trees**
- Operations on Binary Search Trees
  - Insertion
  - Deletion
  - Finding Max and Minimum
  - Finding the k^{th} Minimum Element

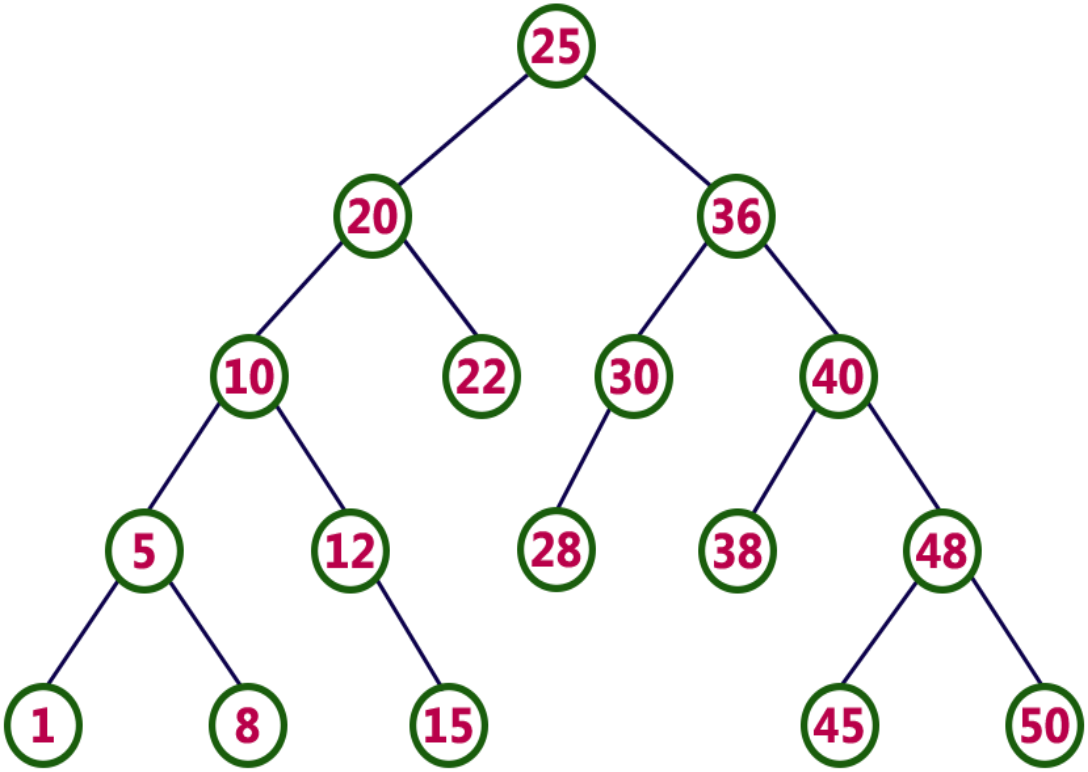
Binary Search Trees

- Difference between Binary Tree and Binary Search Trees
  - In a binary tree, every node can have a maximum of two children but there is no need to maintain the order of nodes basing on their values.
  - In a binary tree, the elements are arranged in the order they arrive at the tree from top to bottom and left to right.
- **Binary Search Tree is a binary tree in which every node contains only smaller values in its left subtree and only larger values in its right subtree.**

Binary Search Trees



Example



Every binary search tree is a binary tree but every binary tree need not to be binary search tree.

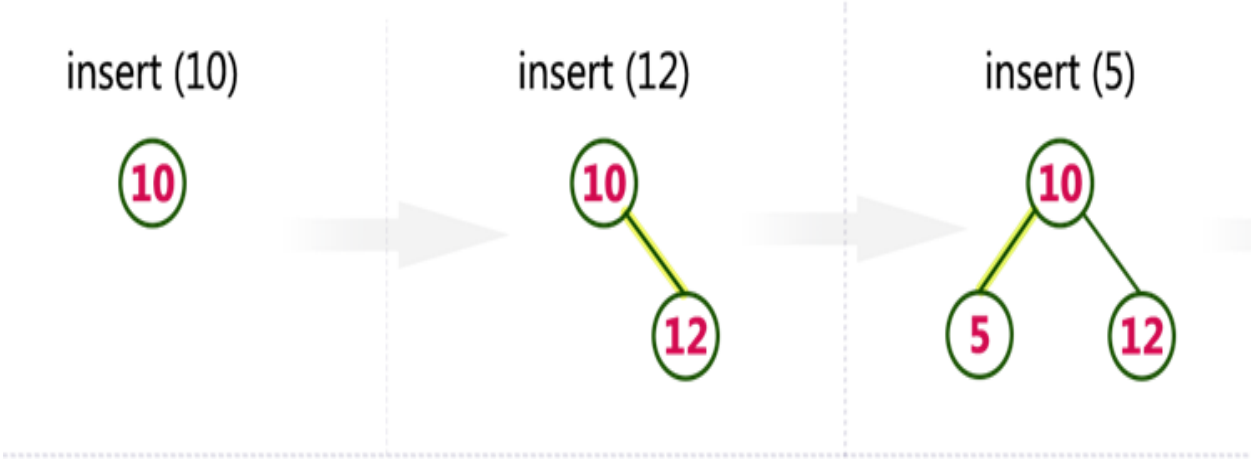
Operations on Binary Search Tree

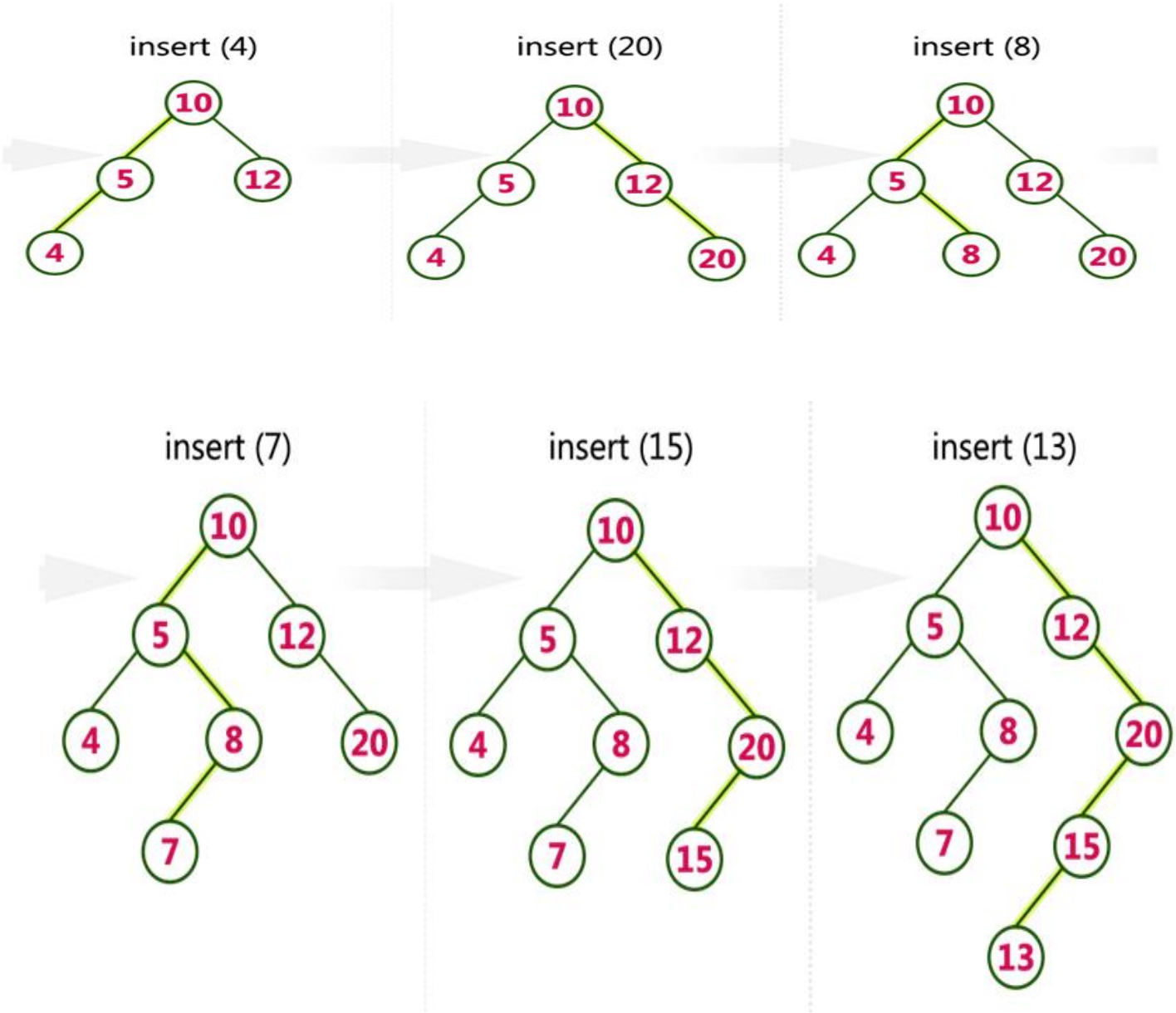
- Insertion
- Deletion
- Searching
  - Finding Max and Minimum
  - Finding the k^{th} Minimum Element

Construction of Binary Search Tree

Construct a Binary Search Tree by inserting the following sequence of numbers...

10,12,5,4,20,8,7,15 and 13




insert (4)
10
5
12
4
insert (20)
10
5
12
4
20
insert (8)
10
5
12
4
8
20
insert (7)
10
5
12
4
8
20
7
insert (15)
10
5
12
4
8
20
7
15
insert (13)
10
5
12
4
8
20
7
15
13

Insertion Operation

- In binary search tree, new node is always inserted as a leaf node
  - Create a **newNode** with given value and set its **left** and **right** to **NULL**.
  - Check whether tree is Empty.
  - If the tree is **Empty**, then set **root** to **newNode**.
  - If the tree is **Not Empty**, then check whether the value of newNode is **smaller** or **larger** than the root node.
  - If newNode is **smaller** than **or equal** to the node then move to its **left** child. If newNode is **larger** than the node then move to its **right** child.
  - Repeat the above steps until reaching **leaf** node
  - After reaching the leaf node, insert the newNode as **left child** if the newNode is **smaller or equal** to that leaf node or else **insert it as right child.**

Structure of Node

```
struct node
{
        int data;
        struct node *left,*right;
};
```

***root=NULL, *tmpnode;**

Creation of new node

```
Createnode(int value)
{
    Struct Node *newnode;
    newnode= malloc(sizeof(Struct  node);
    newnode->data=value;
    newnode->left=NULL;
    newnode->right=NULL;
}
```

Insertion Operation

```
insert(struct * root, int value)
{
        if(value == root->data)
                print "Duplicate Elements";
        else if(value < root->data)
        {
                if (root->left == NULL)
                        root->left = newnode;
                else
                        insert(root->left, value);
        }
        else
        {
                if(root->right == NULL)
                        root->right = newnode;
                else
                        insert(root->right, value);

        }
}
```

Root and value to be inserted is passed every time

Search Operation

```
SEARCH(KEY)
{
 struct node *temp = root;
 while (temp != NULL)
    {
          if (key == temp->data)
                    return 1;
          else if (key > temp->data)
          {
                    temp = temp->right;
          }
          else
          {
                    temp = temp->left;
          }
    }
return 0;
}
```

Finding Minimum

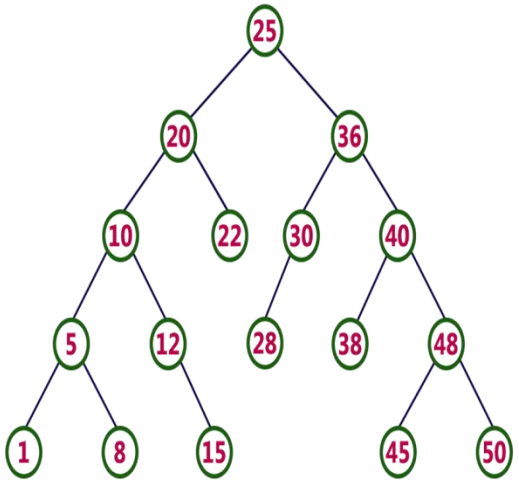
```
struct node *smallest_node(struct node *root)
{
        struct node *q = root;
        while (q != NULL && q->left != NULL)
        {
                q = q->left;
        }
        return q;
}
```

Finding Maximum

```
struct node *largest_node(struct node *root)
{
        struct node *q = root;
        while (q!= NULL && q->right != NULL)
        {
                q = q->right;
        }
        return q;
}
```

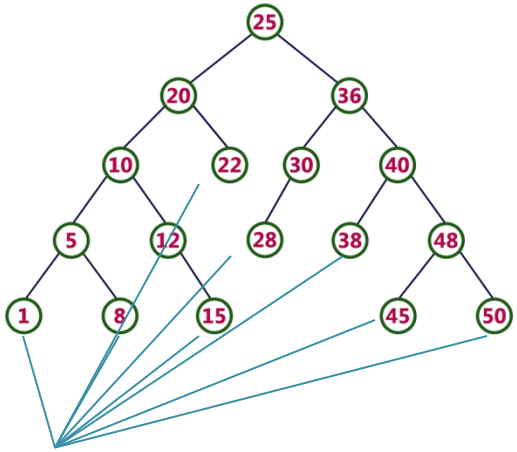
Deletion Operation

- In binary search tree, Deletion is carried out in Three cases
  - Case 1: Deleting a Leaf node (A node with no children)
  - Case 2: Deleting a node with one child
  - Case 3: Deleting a node with two children



Deletion Operation – CASE 1

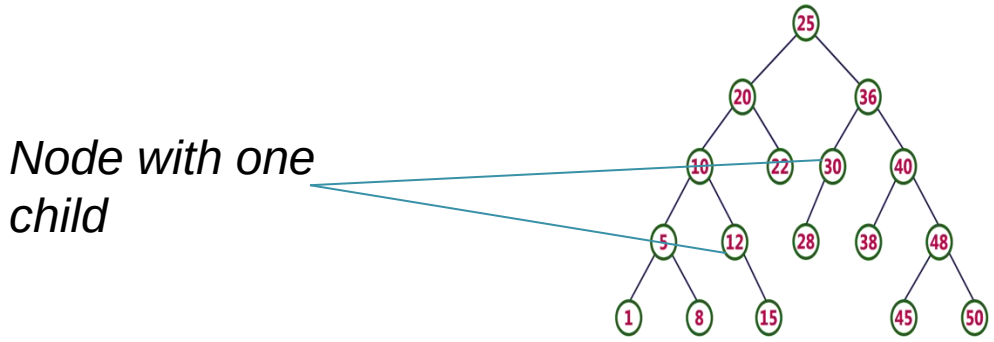
- Case 1: Deleting a Leaf node (A node with no children)
  - Step 1 - Find the node to be deleted using search operation
  - Step 2 - Delete the node using free function (If it is a leaf) and terminate the function.



Node with no children

Deletion Operation – CASE II

- **Case 2: Deleting a node with one child**
  - Step1: Find the node to be deleted using search operation
  - Step 2 : If it has only one child then create a link between its parent node and child node.
  - Step 3: Delete the node using free function and terminate the function.



Deletion Operation – CASE III

- **Case 3: Deleting a node with two children**
  - Step 1 - Find the node to be found using search operation
  - Step 2 - If it has two children, then find the largest node in its left subtree (OR) the smallest node in its right subtree.
  - Step 3 - Swap both deleting node and node which is found in the above step.
  - Step 4 - Then check whether deleting node came to case 1 or case 2 or else go to step 2
  - Step 5 - If it comes to case 1, then delete using case 1 logic.
  - Step 6- If it comes to case 2, then delete using case 2 logic.
  - Step 7 - Repeat the same process until the node is deleted from the tree.

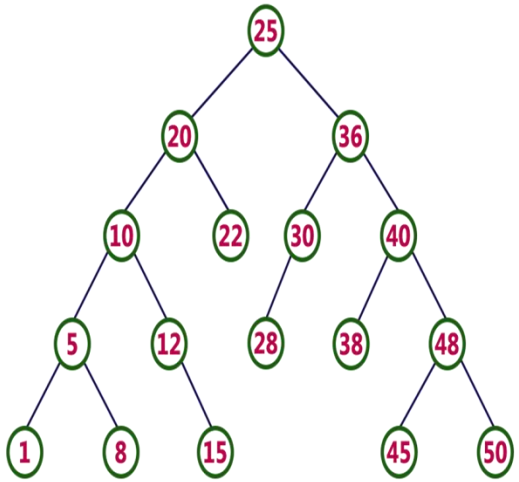
```
deleteNode(struct node* root, int key)
{
    if (root == NULL)
    return root;
    // key less than value of root then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    // key greater than value of root then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    // key is same as value of root, node be deleted is found
    else
    {
```

```
// node with only one child or no child
    if (root->left == NULL)
    {
            struct node* temp = root->right;
            free(root);
            return temp;
    }
    else if (root->right == NULL)
    {
            struct node* temp = root->left;
            free(root);
            return temp;
    }
```

```
    // node with two children, Get smallest in the right subtree)
    struct node* temp = minValueNode(root->right);
    root->key = temp->key;
    // Delete the node
    root->right = deleteNode(root->right, temp->key);
  }
    return root;
}
```

Practice Yourself

- Finding the k^{th} Minimum Element



Example: 5^{th} Minimum Element in the Tree?? 12

8^{th} Minimum Element in the Tree?? 22